학습 노트: 정적 분석 도구 (Pylint / Bandit / Semgrep / radon / ESLint)

작성일: 2026-05-24 관련 파일: `dev2/tools/python/`, `dev2/tools/javascript/`, `dev2/schema.py` 복습 예정: D+1, D+3, D+7 관련 **Task**: Task 3 — CLI 도구 + 다국어 라우터 구현

한 줄 요약

코드를 실행하지 않고 "읽기만" 해서 버그·보안 취약점·복잡도 문제를 찾아내는 검사 도구들. Dev2가 PR 코드를 자동 리뷰할 때 쓴다.

핵심 개념: 정적 분석이란?

(1) 비유

- 동적 분석(실행해서 테스트): 음식을 실제로 먹어보고 맛을 확인 → 정확하지만 시간이 걸리고, 안 먹어본 메뉴는 모름
- 정적 분석(코드를 읽어서 검사): 레시피만 보고 "설탕 1kg은 너무 많다", "이 재료 조합은 식중독 위험"이라고 판단 → 빠르고, 실행 전에 미리 잡음

(2) 명확한 개념

> 정적 분석(Static Analysis)은 프로그램을 실행하지 않고 소스 코드 자체를 분석해 문법 오류, 코드 스멜, 보안 취약점, 복잡도 등을 찾아내는 기법이다. "정적(static)"은 코드가 멈춰있는(실행 안 된) 상태에서 검사한다는 뜻.

(3) 왜 쓰나

- 없으면: 사람이 모든 PR을 한 줄씩 눈으로 읽어야 함 → 느리고, SQL 인젝션 같은 패턴을 놓침
- 있으면: PR이 올라오자마자 자동으로 수백 개 규칙을 검사 → 사람은 진짜 중요한 판단에만 집중

도구 1. Pylint — Python 코드 품질

개념

- 무엇을: 미사용 변수, 네이밍 규칙 위반, 잠재적 버그, 문법 오류
- 언어: Python 전용
- **Dev2 severity**: 대부분 Minor (코드 스멜), error 타입은 Critical

코드

```
pylint src/auth.py --output-format=json
```

```
[{"type": "warning", "line": 10, "message": "Unused variable 'x'", "symbol":
"unused-variable"}]
```

severity 매핑 (`dev2/tools/python/pylint_tool.py`):

```
{"error": "Critical", "warning": "Major", "convention": "Minor", "refactor":
"Minor"}
```

도구 2. Bandit — Python 보안 취약점

개념

- 무엇을: SQL 인젝션, 하드코딩된 비밀번호, `eval()`·`pickle` 같은 위험 함수 사용
- 언어: Python 전용
- **Dev2 severity**: HIGH → Critical, MEDIUM → Major, LOW → Minor (CLAUDE.md §4 Critical=즉시 Slack 에스컬레이션)

코드

```
bandit -r src/ -f json
```

```
{"issue_severity": "HIGH", "issue_text": "Possible SQL injection ...",
"line_number": 42, "filename": "src/auth.py"}
```

도구 3. Semgrep — 다국어 패턴 기반 보안 스캔

개념

- 무엇을: OWASP Top 10 패턴(XSS, CSRF, 인증 미흡 등)을 룰셋으로 검사
- 언어: 언어 무관 (Python, JS, Java, Go, C# 등) + 커스텀 룰 작성 가능
- **Bandit**과 차이: Bandit은 Python만, Semgrep은 여러 언어 + Bandit이 못 잡는 패턴까지 이중 검사

코드

```
semgrep --config=p/owasp-top-ten src/ --json
```

> Dev2 다국어 라우터에서의 의미: .py·.js·.cs 어떤 파일이든 Semgrep 하나로 보안 검사가 가능해, 언어별 도구가 없는 빈틈을 메우는 "공통 보안 그물" 역할.

도구 4. radon — Python 순환복잡도(Cyclomatic Complexity)

개념

- 무엇을: 함수의 분기(if/for/while) 개수를 세서 복잡도를 측정. 숫자가 클수록 이해·테스트가 어려움
- 언어: Python 전용
- **Dev2** 기준: 복잡도 > 10 ⟶ Major, > 20 ⟶ Critical

코드

```
radon cc src/auth.py -j
```

```
복잡도 5  → A 등급 (간단)
복잡도 15 → C 등급 (복잡, 리팩토링 권고)
복잡도 25 → F 등급 (매우 복잡, 즉시 분리)
```

의미: "이 함수는 너무 복잡해서 버그가 숨기 쉽고 테스트하기 어렵다"는 경고.

도구 5. ESLint — JavaScript/TypeScript 코드 품질

개념

- 무엇을: JS/TS의 미사용 변수, == vs === 오용, 위험한 패턴, 스타일 위반
- 언어: JavaScript / TypeScript 전용 (Python의 Pylint에 해당)
- **Dev2** 위치: `dev2/tools/javascript/eslint_tool.py`에서 호출

코드

```
eslint src/app.js --format json
```

```
[{"filePath": "src/app.js", "messages": [
  {"ruleId": "no-unused-vars", "severity": 2, "line": 5, "message": "'x' is
assigned but never used"}
]}]
```

severity 매핑: ESLint의 `severity: 2`(error) ⟶ Major, `severity: 1`(warning) ⟶ Minor

---

도구별 역할 한눈에 보기

도구	검사 대상	언어	Dev2 severity 기준	tools 경로
Pylint	코드 품질/스타일	Python	error→Critical, warning→Major	`tools/python/`
Bandit	보안 취약점	Python	HIGH→Critical, MEDIUM→Major	`tools/python/`
radon	순환복잡도	Python	>10→Major, >20→Critical	`tools/python/`
ESLint	코드 품질/스타일	JS/TS	error→Major, warning→Minor	`tools/javascript/`
Semgrep	OWASP 패턴 보안	다국어	룰셋 기준	`tools/common/`

패턴 정리: 언어별 "품질 도구"(Pylint/ESLint)는 따로 두고, "보안 도구"는 언어별(Bandit)과 공통(Semgrep)을 함께 써서 빈틈을 메운다.

3rd-Project-sub2 프로젝트에서

파일: `dev2/tools/python/`, `dev2/tools/javascript/`, `dev2/tools/common/`, 결과 변환은 `dev2/schema.py`의 `Issue`/`Summary`/`AgentOutput`

역할: 1. 각 도구를 subprocess로 실행해 JSON 원본 결과를 받음 (`Dev2State.static_raw`, `Dev2State.security_raw`) 2. 도구마다 다른 severity 표현(HIGH/error/숫자2)을 Dev2 공통 기준(Critical/Major/Minor)으로 통일 3. 최종적으로 Output Contract A(`AgentOutput`)로 묶어 Dev4 평가 에이전트에게 전달 4. Critical 보안 이슈가 1건이라도 나오면 CLAUDE.md §6에 따라 즉시 Slack 에스컬레이션

연관 개념

- **subprocess / asyncio** → 이 도구들을 코드에서 비동기로 실행 (`docs/learning/ai/subprocess_async.md`)
- 다국어 라우터 → 파일 확장자를 보고 어떤 도구를 돌릴지 결정 (`docs/learning/ai/multilang_router.md`)
- **Output Contract** → 도구별 제각각 결과를 Dev4가 이해하는 공통 형식으로 변환 (`schema.py` `AgentOutput`)
- **OWASP Top 10** → 가장 흔한 웹 보안 취약점 10가지 목록. Semgrep 룰셋의 기준

복습 퀴즈 (3문항)

1. (이해) 정적 분석이 동적 테스트보다 PR 단계에서 유리한 이유 두 가지는?
   - 답: (1) 코드를 실행하지 않고 읽기만 해서 빠르다. (2) 실행 경로를 타지 않은 코드(예: 예외 상황 분기)의 취약점도 패턴으로 찾아낼 수 있다.
2. (비교) Bandit과 Semgrep은 둘 다 보안 도구인데 왜 둘 다 쓰는가?
   - 답: Bandit은 Python 전용으로 깊게 검사하고, Semgrep은 언어 무관(JS·C# 등)이라 다국어 PR에서도 동작하며 OWASP 룰셋으로 Bandit이 못 잡는 패턴까지 이중 검사한다. 서로의 빈틈을 메운다.
3. (적용) Pylint는 `type: "error"`, Bandit은 `issue_severity: "HIGH"`, ESLint는 `severity: 2`로 결과를 준다. Dev2가 이들을 Dev4에 넘기기 전에 반드시 하는 일은?
   - 답: 도구마다 다른 severity 표현을 Dev2 공통 기준(Critical/Major/Minor)으로 통일 매핑하고, Output Contract A(`AgentOutput`) 형식으로 변환한다. 그래야 Dev4가 도구 종류와 무관하게 일관되게 3-Way 판정을 내릴 수 있다.